

Lab Assignment 9.5

1. String Utilities Function:

Consider the following Python function:

```
def reverse_string(text):  
  
    return text[::-1]
```

Task:

1. Write documentation in:
 - a. Docstring
 - b. Inline comments
 - c. Google-style documentation
2. Compare the three documentation styles.
3. Recommend the most suitable style for a utility-based string library.

(a) Docstring Documentation:

```
week 9 > reverse.py > reverse_string  
1  def reverse_string(text):  
2      """  
3      This function takes a string as input and returns the string in reverse order.  
4      Parameters:  
5      text (str): The string to be reversed.  
6      Returns:  
7      str: The reversed string.  
8      Example:  
9      >>> reverse_string("Hello, World!")  
10     '!dlroW ,olleH'  
11     """  
12     return text[::-1]
```

(b) Inline Comments

```
week 9 > reverse.py > ...  
15  def reverse_string(text):  
16      #reverse the string using slicing  
17      return text[::-1]  
18
```

c) Google-Style Documentation

```

week 9 > reverse.py > reverse_string
19 def reverse_string(text : str ) -> str:
20     """
21     #write google style documentation for this function
22     This function takes a string as input and returns the string in reverse order.
23     Args:
24     |     text (str): The string to be reversed.
25     Returns:
26     |     str: The reversed string.
27     """
28     return text[::-1]

```

Comparison of Documentation Styles :

Style	Pros	Cons
Docstring	Simple, readable	Less standardized
Inline Comments	Easy to write	Limited explanation
Google-Style	Clear & professional	Slightly verbose

Google-style documentation is best for **string utility libraries** because it is structured and widely accepted in professional projects

2. Password Strength Checker

Consider the function:

```

def check_strength(password):
    return len(password) >= 8

```

Task:

1. Document the function using docstring, inline comments, and Google style.
2. Compare documentation styles for security-related code.
3. Recommend the most appropriate style.

(a) Docstring

```

week 9 > strength.py > check_strength
1  def check_strength(password):
2      """checks the strength of a password based on length.
3      parameters -----
4      password: str
5      the password to be checked
6      returns -----
7      bool    True if the password is strong, False otherwise
8      """
9      return len(password) >= 8

```

(b) Inline Comments

```

week 9 > strength.py > check_strength
10
11  def check_strength(password):
12      #checks the password is greater than or equal to 8 characters
13      return len(password) >= 8

```

(c) Google-Style Documentation

```

week 9 > strength.py > check_strength
15  def check_strength(password : str) -> bool:
16      """
17      This function checks the strength of a password based on its length.
18      A password is considered strong
19      if it is at least 8 characters long.
20      Args:
21      |    password (str): The password to be checked.
22      Returns:
23      |    bool: True if the password is strong, False otherwise.
24      """
25
26      return len(password) >= 8

```

Google-style documentation is best for **security-related code**, as it clearly explains validation rules and return values

3. Math Utilities Module

Task:

1. Create a module `math_utils.py` with functions:
 - a. `square(n)`
 - b. `cube(n)`
 - c. `factorial(n)`
2. Generate docstrings automatically using AI tools.
3. Export documentation as an HTML file.

> square(n)

```
week 9 > math_utils.py > cube
1  def square(n):
2      """
3      This function takes a number as input and returns the square of the number.
4      Parameters:
5      n (int or float): The number to be squared.
6      Returns:
7      int or float: The square of the input number.
8      """
9      return n ** 2
```

>>> cube(n)

```
week 9 > math_utils.py > cube
11 def cube(n):
12     """
13     This function takes a number as input and returns the cube of the number.
14     Parameters:
15     n (int or float): The number to be cubed.
16     Returns:
17     int or float: The cube of the input number.
18     """
19     return n ** 3
```

>>> factorial(n)

```
week 9 > math_utils.py > factorial
21 def factorial(n):
22     """
23     This function takes a non-negative integer as input and returns its factorial.
24     Parameters:
25     n (int): The non-negative integer to compute the factorial of.
26     Returns:
27     int: The factorial of the input number.
28     """
29     if n == 0:
30         return 1
31     else:
32         return n * factorial(n-1)
```

>>

```
PS C:\Users\User\OneDrive\Desktop\AIAC> cd "week 9"
PS C:\Users\User\OneDrive\Desktop\AIAC\week 9> python -m pydoc math_utils
Help on module math_utils:

NAME
    math_utils

FUNCTIONS
    cube(n)
        This function takes a number as input and returns the cube of the number.
    -- More --
```

>>

```
PS C:\Users\User\OneDrive\Desktop\AIAC\week 9> python -m pydoc -w math_utils
wrote math_utils.html
PS C:\Users\User\OneDrive\Desktop\AIAC\week 9> python -m pydoc -p 1234
Server ready at http://localhost:1234/
Server commands: [b]rowser, [q]uit
server> b
```

math_utils

Functions

cube(n)

This function takes a number as input and returns the cube of the number.

Parameters:

n (int or float): The number to be cubed.

Returns:

int or float: The cube of the input number.

factorial(n)

This function takes a non-negative integer as input and returns its factorial.

Parameters:

n (int): The non-negative integer to compute the factorial of.

Returns:

int: The factorial of the input number.

square(n)

This function takes a number as input and returns the square of the number.

Parameters:

n (int or float): The number to be squared.

Returns:

int or float: The square of the input number.

4. Attendance Management Module

Task:

1. Create a module attendance.py with functions:
 - a. mark_present(student)
 - b. mark_absent(student)
 - c. get_attendance(student)
2. Add proper docstrings.
3. Generate and view documentation in terminal and browse

>>> mark_present(student)

week 9 > attendance.py > mark_present

```
1 def mark_present(student):
2     """
3     Marks a student as present.
4     Args:
5         student (str): The name of the student to be marked as present.
6     Returns:
7         str: A message confirming the student has been marked as present.
8     """
9     return f"{student} has been marked as present."
```

>>> mark_absent(student)

week 9 > attendance.py > mark_absent

```
10 def mark_absent(student):
12     Marks a student as absent.
13     Args:
14         student (str): The name of the student to be marked as absent.
15     Returns:
16         str: A message confirming the student has been marked as absent.
17     """
18     return f"{student} has been marked as absent."
```

>>> get_attendance(student)

week 9 > attendance.py > get_attendance

```
19 def get_attendance(student):
20     """
21     Retrieves the attendance status of a student.
22     Args:
23         student (str): The name of the student whose attendance status is to be retrieved.
24     Returns:
25         str: A message indicating the attendance status of the student.
26     """
27     # For demonstration purposes, we'll return a placeholder message.
28     # In a real implementation, this function would check a database or data structure to get the actual attendance status.
29     return f"The attendance status of {student} is currently unavailable."
```

>>>

```
PS C:\Users\User\OneDrive\Desktop\AIAC> cd "week 9"
PS C:\Users\User\OneDrive\Desktop\AIAC\week 9> python -m pydoc -w attendance
wrote attendance.html
PS C:\Users\User\OneDrive\Desktop\AIAC\week 9> python -m pydoc -p 1234
Server ready at http://localhost:1234/
Server commands: [b]rowser, [q]uit
server> b
```

>>>

attendance

Functions

```
get_attendance(student)
    Retrieves the attendance status of a student.
    Args:
        student (str): The name of the student whose attendance status is to be retrieved.
    Returns:
        str: A message indicating the attendance status of the student.

mark_absent(student)
    Marks a student as absent.
    Args:
        student (str): The name of the student to be marked as absent.
    Returns:
        str: A message confirming the student has been marked as absent.

mark_present(student)
    Marks a student as present.
    Args:
        student (str): The name of the student to be marked as present.
    Returns:
        str: A message confirming the student has been marked as present.
```

5. File Handling Function

Consider the function:

```
def read_file(filename):
    with open(filename, 'r') as f:
        return f.read()
```

Task:

- Write documentation using all three formats.
- Identify which style best explains exception handling.
- Justify your recommendation.

>>> Docstring

```
week 9 > read_file.py > ...
1  def read_file(filename):
2      """
3      Reads and returns the content of a file.
4      parameters -----
5      | filename (str): The name of the file to be read.
6      | returns -----|
7      | str: The content of the file.
8      | """
9      with open(filename, 'r') as f:
10         data = f.read()
11         return data
```

>>> Inline Comments

week 9 > read_file.py > ...

```
13 def read_file(filename):
14     #reads the content of a file and returns it as a string
15     with open(filename, 'r') as f:
16         data = f.read()
17     return data
18
```

>>> Google-Style Documentation

week 9 > read_file.py > ...

```
18
19 def read_file(filename : str) -> str:
20     """
21     This function reads the content of a file and returns it as a string.
22     Args:
23         filename (str): The name of the file to be read.
24     Returns:
25         str: The content of the file.
26     """
27     with open(filename, 'r') as f:
28         data = f.read()
29     return data
```